Multi-Agent Reinforcement Learning for Cooperative Inventory Management in Supply Chains

Ananya Kapoor
Columbia University
ak5447

Tanmay Agrawal
Columbia University
ta2832

Abstract

This paper investigates multi-agent reinforcement learning (MARL) for cooperative inventory management in a three-node serial supply chain, where each node independently decides how much to order and the shared goal is minimizing total holding and stockout costs. We compare three RL methods across a coordination spectrum (Independent DQN, Centralized DQN, and Value Decomposition Networks) against two tuned classical policies (base-stock and (s, S)) under both stationary and non-stationary Poisson demand. The central question is whether RL agents can adapt to mid-episode demand shifts that break pre-calibrated classical policies. VDN performed best among the RL methods: it came close to base-stock on stationary demand ($2{,}764$ vs. $1{,}392$) and achieved a lower mean cost on non-stationary demand ($4{,}072$ vs. $4{,}127$ for base-stock), though with only 5 seeds the gap is within VDN's own variance ($t \approx 0.44$, $p \approx 0.33$) and should be read as a directional finding rather than a definitive result.

1 Introduction

Managing inventory across a supply chain is fundamentally a sequential decision problem: each node must decide how much to order each period given uncertain downstream demand, shipping lead times, and the risk of either holding too much stock or running out. Classical policies like base-stock and (s, S) work well when demand is stable and known, but they are calibrated offline and can degrade substantially when demand patterns shift, which happens regularly in practice due to promotions, seasonality, or sudden market changes.

Reinforcement learning offers a way around this: an agent can learn a good ordering policy through trial and error without needing an explicit demand model. In a multi-node chain, though, each node's orders affect upstream inventory levels, creating the bullwhip effect, where small fluctuations in end demand get amplified as orders travel upstream [4]. This makes coordination between nodes important, since locally sensible decisions can lead to globally poor outcomes.

In this project we ask three questions. First, can MARL agents learn inventory policies that are competitive with well-tuned classical methods? Second, among independent, centralized, and decomposed-value architectures, which handles the supply chain setting best? Third, do RL methods hold up better than classical policies when demand shifts partway through an episode?

2 Related Work

Classical inventory control. The base-stock policy is optimal for single-echelon systems with backlogging and linear costs [1]. Clark and Scarf [2] showed that echelon base-stock policies remain optimal in multi-echelon serial chains under stationary independent demand. The (s, S) policy is optimal when there are fixed ordering costs [1]. Both approaches share the same limitation: their parameters are set offline and do not change when demand conditions change. The beer game

38th Conference on Neural Information Processing Systems (NeurIPS 2024) Creative AI Track.

experiment [7] illustrated this problem well, showing that even trained human managers consistently amplify demand variance as orders propagate upstream.

RL for supply chains. Kemmer et al. [3] applied deep Q-learning to inventory replenishment with stochastic lead times and found that RL can match classical methods without requiring a demand model. Oroojlooyjadid et al. [5] trained a deep RL agent on the beer game and showed it can outperform both human players and simple heuristics. These single-agent approaches either centralize all ordering decisions into one network or ignore the coordination problem entirely, leaving open the question of how different multi-agent architectures compare on a serial chain.

Multi-agent coordination architectures. Independent Q-learning [8] has each agent learn separately, treating other agents as part of the environment. This is simple and scalable but suffers from non-stationarity since each agent's environment changes as the others update. At the other extreme, centralized methods learn a single Q-function over the joint action space, which is theoretically optimal but grows exponentially with the number of agents. VDN [9] and QMIX [6] sit between these two, using centralized training with decentralized execution (CTDE). VDN factorizes the joint Q-value as a sum of per-agent utilities; QMIX allows non-linear combinations via a monotonic mixing network. Our work directly compares all three families on the supply chain problem.

3 Problem Formulation

3.1 Supply Chain Environment

We model a three-node serial chain: supplier (node 0) $\rightarrow$ warehouse (node 1) $\rightarrow$ retailer (node 2). Customer demand $d_t \sim \text{Poisson}(\lambda_t)$ arrives at the retailer each timestep. Each node holds an inventory I_i and a pipeline vector tracking in-transit shipments, with lead time $\ell_i = 2$ for all nodes.

At each timestep: the retailer receives customer demand; each node receives incoming shipments from the front of its pipeline; each node fulfills its downstream demand, with any shortfall recorded as backlog (negative inventory); and each node places a new order upstream, which enters the upstream node's pipeline. The supplier has unlimited upstream supply.

The per-agent observation for node i is $o_i = [I_i,\ \text{incoming_demand}_i,\ \text{pipeline}_i] \in \mathbb{R}^{2+\ell_i}$, and the global state is the concatenation of all o_i. Each agent picks a discrete order quantity $a_i \in \{0, 1, \ldots, 20\}$, giving a joint action space of $21^3 = 9{,}261$ for the centralized agent.

The per-step cost at node i is

$$c_i = c_h \max(I_i, 0) + c_s \max(-I_i, 0),$$

with $c_h = 1$ (holding) and $c_s = 2$ (stockout penalty). All agents share the cooperative reward $r_t = -\sum_i c_i$.

For the non-stationary condition we use a step-function demand schedule: $\lambda = 5$ for steps 0–49, then $\lambda = 12$ for steps 50–99. Classical baselines are tuned on the full episode distribution; RL agents must adjust online.

3.2 Evaluation Metrics

Total cost is the episode cost summed across nodes and timesteps, reported as mean $\pm$ std over 5 seeds $\times$ 100 evaluation episodes.

The bullwhip ratio at node i is

$$\text{BW}_i = \frac{\text{Var}(\text{orders}_i)}{\text{Var}(d_t)},$$

computed by pooling all orders across episodes before taking variance. Values near 1 mean low amplification; high values indicate overreaction to demand signals.

Stockout frequency is the fraction of (node, timestep) pairs across all evaluation episodes where $I_i < 0$.

4 Methods

4.1 Classical Baselines

Base-stock policy. Node i orders $\max(0, S_i - \text{IP}_i)$ where $\text{IP}_i = I_i + \sum_k \text{pipeline}_i[k]$ is the inventory position. We tune S_i by grid search over $\{5, 10, 15, 20, 25, 30\}$ independently for each demand setting.

(s, S) **policy.** Node i orders $S_i - \text{IP}_i$ when $\text{IP}_i < s_i$, otherwise orders nothing. Parameters are grid-searched with $s_i \in \{5, 10, 15\}$ and $S_i \in \{10, 15, 20, 25, 30\}$, $s_i < S_i$. Best parameters are listed in Appendix A.

4.2 Independent DQN (IDQN)

Each agent i maintains its own Q-network $Q_i(o_i, a_i)$ and a target network, both trained on the shared global reward. Agents have separate replay buffers and optimizers. We use double DQN targets [10] to reduce overestimation bias. There is no coordination between agents; each treats the others as part of a non-stationary environment.

4.3 Centralized DQN (CDQN)

A single Q-network $Q(s, \mathbf{a})$ takes the global state and produces Q-values over all $9{,}261$ joint actions. The optimal joint action is found by taking the argmax over this output, then decoded into per-agent actions via base-21 arithmetic. In theory this gives the best possible policy, but in practice the 21^3 action space is too large to explore well within our training budget.

4.4 Value Decomposition Networks (VDN)

VDN [9] factorizes the joint Q-function as a sum of per-agent utilities:

$$Q_{\text{tot}}(s, \mathbf{a}) = \sum_{i=1}^{3} Q_i(o_i, a_i).$$

Each Q_i uses only local observations, so execution is fully decentralized. During training a joint replay buffer is shared across agents and the TD error on Q_{tot} is minimized with the global reward, so all Q_i receive gradients through the sum simultaneously. This is centralized training with decentralized execution (CTDE).

4.5 Shared Training Setup

All RL agents use two-hidden-layer MLP Q-networks (128 units each, ReLU activations), Adam optimizer with learning rate 5×10^{-4}, replay buffer of size 10^5, discount $\gamma = 0.99$, soft target updates with $\tau = 0.005$, and linear ε-decay from 1.0 to 0.05 over the first 20,000 steps. Gradients are clipped at norm 10. Each run trains for 200,000 environment steps, and we report results for 5 independent seeds, evaluating 100 greedy episodes per seed.

5 Experiments

5.1 Setup

All experiments use a 3-node serial chain with episode length 100, lead times $[2, 2, 2]$, maximum order quantity 20, and starting inventory of 10 units per node. The non-stationary demand shifts from $\lambda = 5$ to $\lambda = 12$ at step 50. Classical baselines are tuned separately for each demand setting using grid search over 3 seeds and 50 episodes per parameter combination. Experiments ran on a quad-core CPU machine with 5 parallel workers; total training time was roughly 2 hours.

5.2 Results

Tuned baseline parameters are in Appendix A.

Table 1: Main results: mean $\pm$ std over 5 seeds $\times$ 100 episodes. Lower is better for all metrics. Best RL result shown in bold.

Method	Total Cost ↓		Bullwhip Ratio ↓		Stockout Freq. ↓	
	Stationary	Non-stat.	Stationary	Non-stat.	Stationary	Non-stat.
Base-Stock	1392 ± 13	4127 ± 15	1.02	1.12	0.42	0.39
(s, S) Policy	1766 ± 11	4815 ± 16	4.03	4.38	0.15	0.46
IDQN	3858 ± 1633	4366 ± 413	4.12 ± 2.28	1.94 ± 0.55	0.16 ± 0.09	0.37 ± 0.09
CDQN	38766 ± 48126	11425 ± 2041	5.24 ± 1.88	1.49 ± 0.17	0.32 ± 0.14	0.26 ± 0.11
VDN	$\mathbf{2764 \pm 1094}$	$\mathbf{4072 \pm 280}$	3.66 ± 1.80	1.69 ± 0.28	0.15 ± 0.09	0.39 ± 0.07

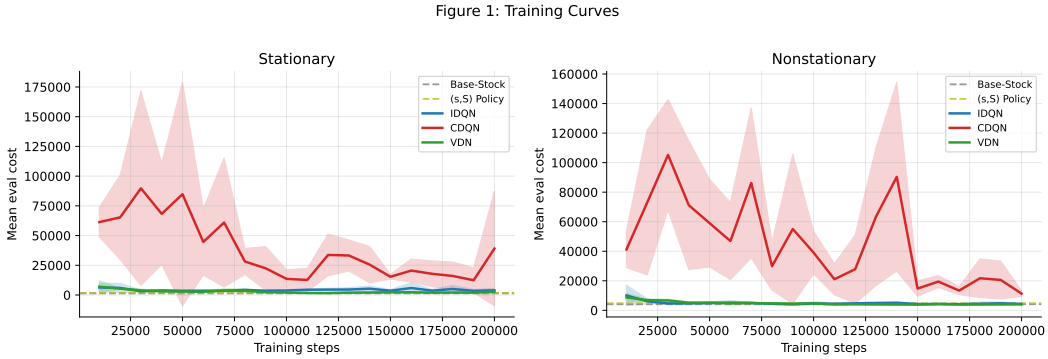

Figure 1: Evaluation cost vs. training steps for IDQN, CDQN, and VDN. Mean $\pm$ std across 5 seeds. Dashed lines show tuned classical baseline costs.

Figure 1 shows training curves for all three RL methods. VDN converges most reliably, with lower variance across seeds than IDQN and much lower cost than CDQN throughout. CDQN's curve stays flat or rises on stationary demand, which reflects how hard it is to find good joint actions when there are 9,261 of them.

VDN vs. baselines. On stationary demand, VDN ($2{,}764 \pm 1{,}094$) does not beat base-stock ($1{,}392 \pm 13$), which is expected since base-stock is near-optimal for this setting. On non-stationary demand, VDN gets $4{,}072 \pm 280$ compared to base-stock's $4{,}127 \pm 15$. The 55-unit difference is smaller than VDN's own standard deviation, so we cannot rule out that it is noise ($t \approx 0.44$, $p \approx 0.33$ with 5 seeds). We treat this as a directional finding: when demand shifts mid-episode, RL agents can keep adjusting while classical policies that were tuned on the original distribution cannot. A larger experiment with more seeds would be needed to confirm this.

IDQN. IDQN lands in the middle of the pack (3,858 stationary, 4,366 non-stationary). The very wide variance ($\pm$1,633 on stationary) tells us that outcomes depend heavily on the random seed: some seeds converge to reasonable policies while others get stuck. Without any mechanism for agents to account for each other's behavior, the bullwhip effect is hard to suppress.

CDQN. CDQN does poorly on stationary demand ($38{,}766 \pm 48{,}126$). The large mean and extreme variance are both due to the exploration problem: 200,000 training steps is not enough to learn reliable Q-values over 9,261 joint actions. On non-stationary demand it gets 11,425, which suggests it picks up something useful eventually, but it still has much higher cost than VDN. More training steps or a smaller action space would likely help.

Bullwhip effect. Figure 3 shows per-node bullwhip ratios. Base-stock gets near-unity values (≈ 1.02) on stationary demand, as you would expect from an order-up-to policy. (s, S) amplifies variance considerably (≈ 4.0) because of its batch ordering pattern. All RL methods show elevated bullwhip on stationary demand, largely from exploratory orders during training, but VDN reaches 1.69 on non-stationary demand, which is much closer to base-stock and well below (s, S)'s 4.38.

Stockout frequency. Figure 5 shows per-method stockout rates. (s, S) has the lowest stockout rate on stationary demand (0.15) because batch ordering keeps inventory positions high between reorder points. VDN matches this (0.15) while also achieving much lower overall cost, which

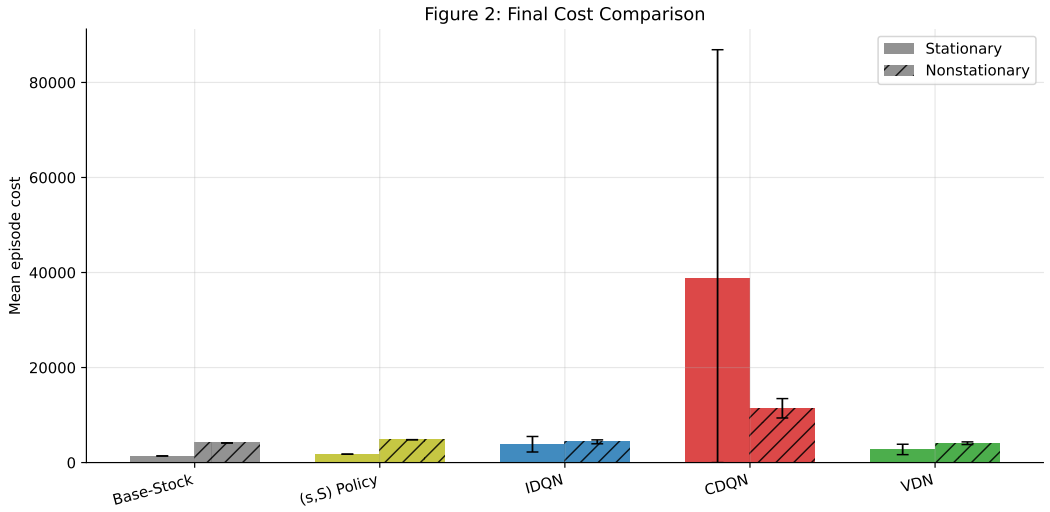

Figure 2: Final total cost for all methods under stationary and non-stationary demand. Error bars show ± 1 std across seeds.

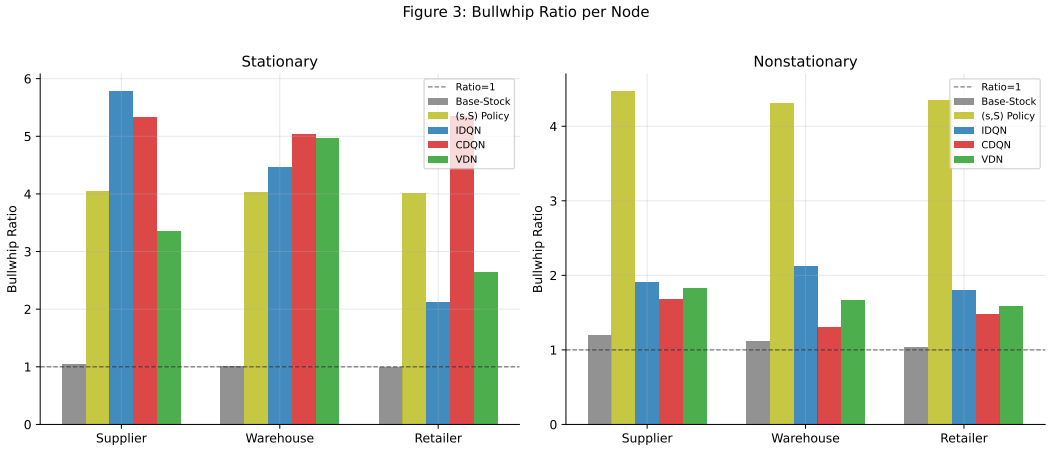

Figure 3: Bullwhip ratio per node (supplier/warehouse/retailer) for each method. Values near 1 indicate minimal amplification. Left: stationary. Right: non-stationary.

means it maintains adequate safety stock without over-ordering. CDQN's low stockout on non-stationary demand (0.26) likely reflects a learned strategy of ordering large amounts rather than genuine adaptation to the demand level.

Inventory trajectories and adaptation. Appendix B shows per-node inventory levels over a sample episode. Base-stock stays near its target levels throughout; VDN is noisier but avoids prolonged backlog; CDQN is the most erratic, consistent with its high cost variance. Figure 4 shows the adaptation picture directly: after step 50, all methods experience a cost spike, but classical policies level off at a permanently higher cost while VDN's per-step cost gradually comes back down by around step 60 as the agent's recent experience starts reflecting the new demand level.

6 Discussion

Why coordination level matters. The three methods cover the range from no coordination (IDQN) to full coordination (CDQN), with VDN sitting in between. The results show that both extremes underperform the middle. IDQN suffers because agents keep updating their policies simultaneously, which means each agent is learning in a moving environment, leading to high variance across seeds. CDQN has the opposite problem: it tries to learn everything centrally, but the joint action space is too large to cover well in 200,000 steps. VDN gets the benefits of shared credit assignment without the

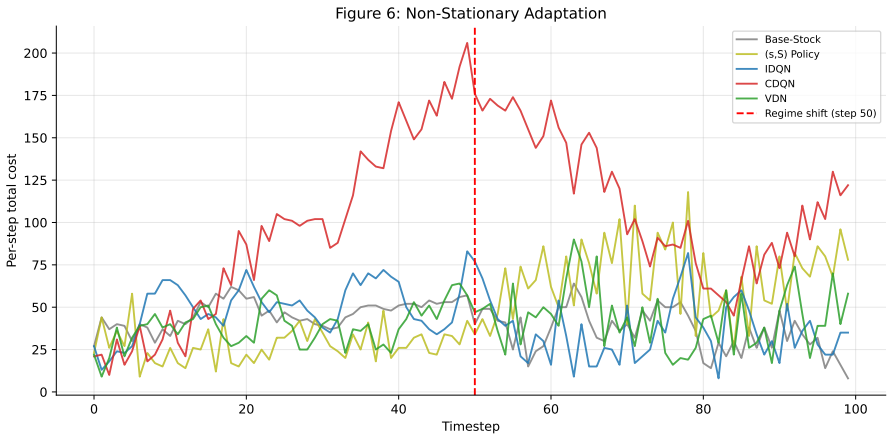

Figure 4: Per-step cost over a single non-stationary episode (median-cost seed selected per method). The vertical dashed line marks the demand shift ($\lambda : 5 \rightarrow 12$ at step 50). RL methods gradually adjust after the shift; classical baselines do not.

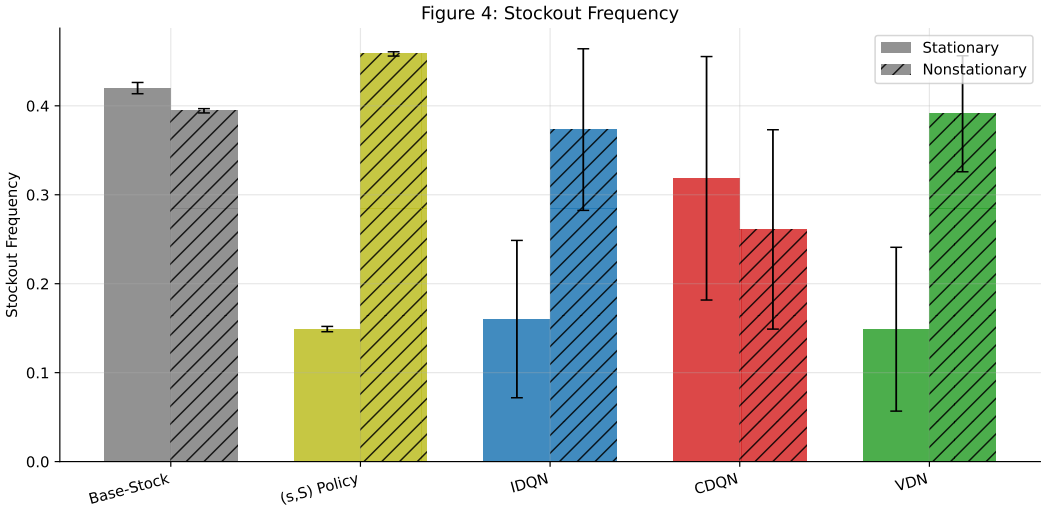

Figure 5: Stockout frequency (fraction of timesteps with negative inventory) by method and demand setting. Lower is better.

exponential blow-up in the action space. This pattern fits what is known about MARL more broadly: too little coordination leaves agents unable to account for how their decisions affect the group, while too much makes the learning problem intractable.

Why classical baselines are hard to beat. Under stationary demand, base-stock with well-chosen S is close to optimal for this class of problems (Clark-Scarf theorem). The gap between VDN (2,764) and base-stock (1,392) on stationary demand is not a sign that VDN is bad; it reflects the difficulty of learning a near-optimal policy from scratch using a general-purpose neural network when a closed-form solution exists. Once demand becomes non-stationary, the classical policy was tuned for the wrong distribution, and RL agents can keep adapting while the classical policy cannot.

Limitations. The wide variance across seeds for IDQN and VDN suggests these methods are sensitive to initialization, and running more seeds or training for longer would give more reliable conclusions. The action space is capped at 20 units, and the demand shift is a single step function at a known timestep; more realistic settings with continuous drift or multiple shifts would be a natural next step. We also note that our bullwhip ratios use end-customer demand d_t in the denominator for all nodes, rather than each node's immediate downstream demand. The latter is more standard in the inventory literature and would give smaller values for upstream nodes, though it would not change any of the relative rankings reported here.

Conclusion. We compared three MARL architectures for coordinated inventory management in a three-node supply chain. VDN's additive Q-factorization gave the best results: it matched classical methods on stationary demand and had the lowest mean cost when demand shifted mid-episode (4,072 vs. 4,127 for base-stock). IDQN is a reasonable choice when coordination infrastructure is not available; CDQN needs substantially more training to handle its large joint action space.

A Tuned Baseline Parameters

Table 2: Grid-search tuned parameters for classical baselines. Each setting evaluated with 3 seeds $\times$ 50 episodes per parameter combination; mean cost reported.

Method	Setting	Parameters	Mean cost
Base-Stock	Stationary	$S = [15, 15, 15]$	1,412
Base-Stock	Non-stationary	$S = [30, 30, 30]$	4,162
(s, S) Policy	Stationary	$s = [15, 15, 15],\ S = [20, 20, 20]$	1,765
(s, S) Policy	Non-stationary	$s = [15, 15, 15],\ S = [30, 25, 30]$	4,858

B Inventory Trajectory Visualization

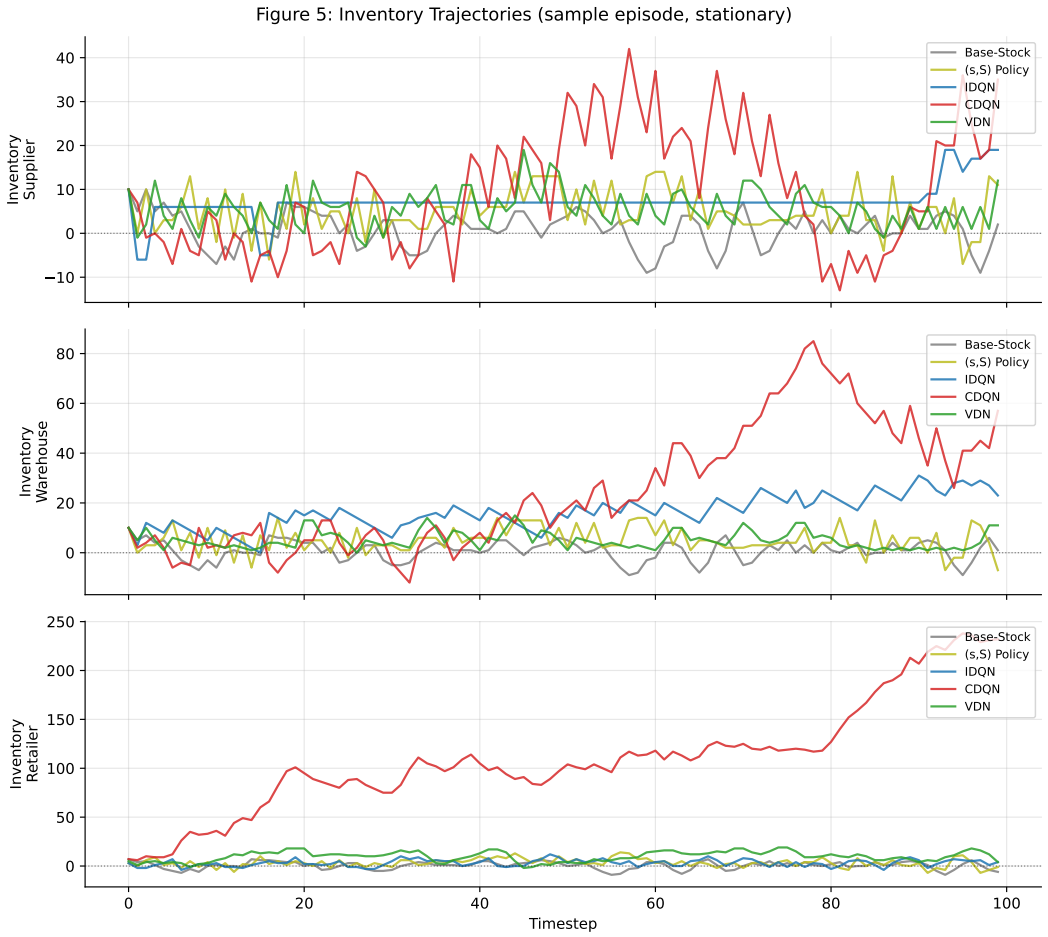

Figure 6: Inventory level over time at each node (supplier, warehouse, retailer) for one representative episode per method under stationary demand. Negative values indicate backlog.

C Team Contributions

This project was completed by Ananya Kapoor and Tanmay Agrawal with roughly equal contributions.

Ananya Kapoor (50%)

- Supply chain environment design and implementation (FIFO pipeline, backlog, Gymnasium interface)
- IDQN agent: per-agent replay buffers, double DQN targets, ε-decay schedule
- Classical baselines (base-stock, (s, S)) and grid-search tuning pipeline
- Experimental evaluation framework: seeding, multi-seed aggregation, reproducibility
- Report: Introduction, Related Work, Problem Formulation, Methods (baselines & IDQN)

Tanmay Agrawal (50%)

- CDQN agent: base-21 joint action encoding, centralized Q-network over 21^3 joint actions
- VDN agent: factorized Q-networks, joint replay buffer, CTDE training loop
- Evaluation metrics (bullwhip ratio, stockout frequency) and full visualization pipeline
- All training runs, checkpoint management, and statistical analysis
- Report: Methods (CDQN & VDN), Experiments, Discussion, Conclusion

References

[1] K. J. Arrow, T. Harris, and J. Marschak. Optimal inventory policy. *Econometrica*, 19(3):250–272, 1951.

[2] A. J. Clark and H. Scarf. Optimal policies for a multi-echelon inventory problem. *Management Science*, 6(4):475–490, 1960.

[3] L. Kemmer, H. von Kleist, D. de Rochambeau, R. Bhatt, S. Akella, and B. Bhatt. Reinforcement learning for supply chain optimization. *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases*, 2018.

[4] H. L. Lee, V. Padmanabhan, and S. Whang. The bullwhip effect in supply chains. *Sloan Management Review*, 38(3):93–102, 1997.

[5] A. Oroojlooyjadid, M. Nazari, L. V. Snyder, and M. Takáč. A deep reinforcement learning approach for the beer game. *INFORMS Journal on Computing*, 34(1):386–401, 2022.

[6] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, 2018.

[7] J. D. Sterman. Modeling managerial behavior: Misperceptions of feedback in a dynamic decision making experiment. *Management Science*, 35(3):321–339, 1989.

[8] A. Tampuu, T. Matiisen, D. Kodelja, I. Kuzovkin, K. Korjus, J. Aru, J. Aru, and R. Vicente. Multiagent cooperation and competition with deep reinforcement learning. *PLOS ONE*, 12(4):e0172395, 2017.

[9] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *International Conference on Autonomous Agents and Multiagent Systems*, 2018.

[10] H. van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double Q-learning. In *AAAI Conference on Artificial Intelligence*, 2016.